

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA-0409

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

CASE STUDY

Code of Conduct:

I HUSSAIN V (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Explain the concepts of logical address and physical address. Describe how logical addresses are translated into physical addresses.

A logical address is the address generated by the CPU when a program executes. It is also called a virtual address because it represents the location of an instruction or data item from the viewpoint of the running process. A process normally does not know the actual location of its data in RAM. The logical address space is therefore separated from the physical memory space.

A physical address is the actual address in main memory (RAM) where an instruction or data item is stored. The Memory Management Unit (MMU), with support from the operating system, performs the translation from logical addresses to physical addresses before the memory is accessed.

The method of translation depends on the memory-management technique. In a simple relocation system, the physical address is obtained by adding the logical address to the base address of the process.

Physical Address = Base Address + Logical Address

For example, assume that a process has a base address of 10,000 and the CPU generates logical address 2,500. The MMU calculates:

Physical Address = 10,000 + 2,500 = 12,500.

Thus, the process continues to use logical address 2,500, while the actual RAM location accessed is physical address 12,500.

This separation is important because it allows processes to be loaded at different locations in memory without changing their program instructions. It also provides memory protection and supports techniques such as paging and segmentation. Therefore, logical-to-physical address translation is a fundamental function of modern operating systems.

2. Explain the complete address translation mechanism using base and limit registers. Explain how this method prevents unauthorized memory access.

Base and limit registers are hardware registers used to perform address translation and protect the memory assigned to a process. They are particularly useful in systems where a process occupies one continuous region of physical memory.

The base register contains the starting physical address of the process. The limit register specifies the size or maximum valid range of the process's logical address space. Whenever the CPU generates a logical address, the hardware first checks whether the address is within the permitted limit.

The basic steps are:

1. The CPU generates a logical address.

2. The logical address is compared with the value in the limit register.
3. If the logical address is less than the limit, it is considered valid.
4. The base value is added to the logical address.
5. The resulting value is the physical address sent to memory.
6. If the logical address is outside the limit, a protection trap is generated.

For example, consider Base = 5,000 and Limit = 2,000. If the CPU generates logical address 1,500:

Physical Address = 5,000 + 1,500 = 6,500.

Since 1,500 < 2,000, the access is allowed.

Now suppose the CPU generates logical address 2,200. Since 2,200 is greater than the limit of 2,000, the access is rejected. The hardware generates a protection exception instead of allowing the process to access memory.

This mechanism prevents a process from accessing the memory of another process or the operating system. It is especially important in a multi-programming environment where several processes are stored in memory simultaneously. Base and limit registers therefore provide both address relocation and basic memory protection.

3. Explain the concept of paging. Describe how logical memory is divided into pages and mapped to physical frames. Discuss its role in avoiding external fragmentation and improving memory utilization.

Paging is a non-contiguous memory-management technique in which the logical address space of a process is divided into fixed-size blocks called pages. Physical memory is also divided into fixed-size blocks of the same size called frames. Because pages and frames have equal sizes, any page can be placed into any available frame.

A logical address generated by the CPU is divided into two parts: the page number and the page offset. The page number identifies which page contains the required data, while the offset identifies the exact position within that page.

The operating system maintains a page table for every process. The page number is used as an index into this table. The page table provides the frame number in which that page is currently stored. The frame number is then combined with the original offset to produce the physical address.

For example, assume a process has page 3 mapped to frame 8. If the logical address refers to page 3 with an offset of 200, the physical address refers to frame 8 with the same offset 200.

Paging is useful because a process does not need to occupy one continuous region of physical memory. Its pages can be placed in different free frames. Therefore,

paging eliminates external fragmentation, which occurs when free memory is divided into many small non-contiguous holes.

Paging also improves memory utilization because unused frames can be allocated to other processes. However, paging can produce internal fragmentation, especially when the final page of a process is only partially filled. Overall, paging provides flexible memory allocation and forms the basis of virtual memory in many operating systems.

4. Explain the structure and working of a page table. Describe how page table entries are used to map logical addresses to physical addresses. Discuss advantages and limitations of paging.

A page table is a data structure maintained by the operating system to translate a process's logical page numbers into physical frame numbers. Each process generally has its own page table because different processes may have different mappings.

A page-table entry (PTE) commonly contains the following information:

- Frame number – identifies the physical frame containing the page.
- Present/valid bit – indicates whether the page is currently in physical memory.
- Protection bits – specify whether the page can be read, written, or executed.
- Reference bit – records whether the page has recently been accessed.
- Dirty bit – indicates whether the page has been modified.

When the CPU generates a logical address, it is divided into a page number and an offset. The page number is used to locate the corresponding entry in the page table. The frame number obtained from the entry is combined with the offset to form the physical address.

For example, suppose logical page 4 is mapped to physical frame 10 and the offset is 300. The requested data will be located in frame 10 at offset 300.

The major advantage of paging is that it eliminates external fragmentation because pages can be placed in any available frames. It also supports virtual memory, allows non-contiguous allocation, and makes memory sharing and protection easier.

However, page tables themselves consume memory. Large processes or large address spaces may require very large page tables. Address translation also introduces additional processing and memory-access overhead. To reduce this overhead, systems use techniques such as the Translation Lookaside Buffer (TLB) and multi-level page tables.

5. Describe the working of the Translation Lookaside Buffer (TLB). Explain how TLB reduces memory access time and improves system performance.

The Translation Lookaside Buffer (TLB) is a small, high-speed associative memory used to store recently used page-table entries. It is located within or close to the Memory Management Unit. Its main purpose is to speed up logical-to-physical address translation.

Without a TLB, every memory access may require two memory accesses: one to read the page-table entry and another to access the actual data. This increases the effective memory access time.

When a logical address is generated, the CPU divides it into a page number and offset. The page number is first searched in the TLB.

If the page number is present, a TLB hit occurs. The TLB immediately supplies the corresponding frame number. The frame number and offset are then used to access the required location in memory. The page table does not need to be accessed for the translation.

If the page number is not present, a TLB miss occurs. The system must access the page table in main memory to find the frame number. The newly obtained mapping may then be stored in the TLB for future accesses.

For example, suppose TLB lookup takes 10 ns and main memory access takes 100 ns. In a simplified case, a TLB hit may take about 110 ns including the memory access, while a TLB miss may require an additional memory access for the page table and take about 210 ns.

Because programs often show locality of reference, recently used pages are likely to be accessed again. Therefore, a high TLB hit ratio significantly reduces average memory access time and improves overall CPU and system performance.

6. Differentiate between TLB hit and TLB miss. Explain how each affects memory access time and system efficiency.

A TLB hit and a TLB miss describe the result of searching the Translation Lookaside Buffer for a page-to-frame mapping.

A TLB hit occurs when the required page number is already present in the TLB. The frame number is obtained directly from the TLB, so the system can quickly construct the physical address and access memory. Since there is no need to access the page table in main memory, a TLB hit results in lower memory access time.

A TLB miss occurs when the required page number is not present in the TLB. The operating system or memory-management hardware must then consult the page table in main memory to obtain the required frame number. The mapping is usually loaded into the TLB so that future references to the same page can be faster.

**Example: Suppose TLB lookup requires 10 ns and memory access requires 100 ns. A simplified TLB hit requires approximately:
 $10 + 100 = 110$ ns.**

**For a TLB miss, the system may require:
 $10 + 100$ (page table) + 100 (data) = 210 ns.**

Therefore, TLB misses increase effective memory access time and reduce system efficiency. A high TLB hit ratio is desirable because it means most address translations can be completed quickly.

In practice, the exact time depends on the processor and memory hierarchy, but the basic principle remains the same: TLB hits are faster, while TLB misses introduce additional translation overhead.

7. Explain how memory protection is achieved using base/limit registers and access permissions. Discuss its importance in a multi-user environment and explain what happens when a process violates access rules.

Memory protection is the mechanism used by an operating system to prevent one process from accessing memory that belongs to another process or to the operating system itself. It is essential for reliability, security, and safe multi-tasking.

Base and limit registers provide one basic form of protection. The base register identifies the starting physical address of a process, while the limit register specifies the maximum valid logical address range. Before an address is used, the hardware checks whether it lies inside the permitted range.

For example, suppose Base = 4,000 and Limit = 3,000. A logical address of 2,000 is valid because it is less than 3,000, and the physical address becomes 6,000. However, a logical address of 3,500 is invalid because it exceeds the permitted limit.

Access permissions provide another layer of protection. A memory region may be marked as read-only, read-write, or executable. For example, program code may be executable and readable but not writable, while data may be readable and writable but not executable.

Protection is especially important in a multi-user or multi-programming environment. Several users and processes may execute at the same time, and one process must not be allowed to modify another process's data or critical operating-system memory.

When a process violates a memory protection rule, the hardware detects the violation and generates a trap or exception. Control is transferred to the operating system, which can terminate the invalid operation, report an error, or

terminate the offending process. Thus, memory protection prevents accidental as well as unauthorized memory access.

8. Describe the concept of segmentation in detail. Explain how programs are divided into logical segments such as code, data, and stack, and discuss the advantages of segmentation over other memory management techniques.

Segmentation is a memory-management technique that divides a program into variable-sized logical units called segments. Unlike paging, where memory is divided into fixed-size blocks, segmentation follows the logical structure of a program.

Common segments include:

- Code segment – contains program instructions.
- Data segment – contains global and static data.
- Stack segment – stores function calls, local variables, and return addresses.
- Heap segment – contains dynamically allocated memory.

A logical address in segmentation consists of two components:

<Segment Number, Offset>

The segment number identifies the required logical segment, while the offset specifies the location within that segment. A segment table stores the base address, limit, and protection information for each segment.

For example, suppose segment 2 represents the data segment and has Base = 12,000 and Limit = 3,000. If the CPU generates <2, 1,500>, the physical address is: $12,000 + 1,500 = 13,500$.

The access is valid because the offset is within the limit.

Segmentation provides several advantages. It matches the programmer's logical view of a program and allows different segments to have different protection rules. For example, the code segment can be read-only, while the data segment can be read-write. Segments can also be shared between processes when appropriate.

However, segmentation uses variable-sized memory blocks and can therefore produce external fragmentation. Compaction may sometimes be required. Paging avoids this problem because it uses fixed-size pages. Thus, segmentation is especially useful for logical organization, protection, and sharing, while paging is generally better for efficient physical-memory allocation.

9. Explain the structure and functioning of a segment table. Describe how logical addresses are translated into physical addresses using segment number and offset.

A segment table is a data structure used by the operating system to translate logical addresses into physical addresses in a segmentation system. Each entry in the table represents one segment of a process.

A typical segment-table entry contains:

- Segment base – the starting physical address of the segment.
- Segment limit – the size or maximum valid offset of the segment.
- Protection information – specifies permitted operations such as read, write, or execute.
- Other control bits – may indicate whether the segment is present or accessible.

A logical address is represented as:

<Segment Number, Offset>

The segment number is used as an index into the segment table. The system retrieves the base address and limit for that segment. The offset is then compared with the limit.

If:

Offset < Limit

the address is valid and the physical address is calculated as:

Physical Address = Segment Base + Offset.

Consider a segment-table entry:

Segment 2 Base = 12,000, Limit = 3,000.

For logical address <2, 1,500>:

Physical Address = 12,000 + 1,500 = 13,500.

Since 1,500 is less than 3,000, the access is permitted.

For logical address <2, 3,500>, the offset is greater than the limit. Therefore, the address is invalid and a protection exception is generated.

The segment table therefore performs two important functions: address translation and memory protection. It also allows different logical sections of a program to have different sizes and access permissions.

10. Compare and contrast paging and segmentation in terms of memory allocation, fragmentation, flexibility, and performance.

Paging and segmentation are two important memory-management techniques, but they organize memory in different ways.

Paging divides a process's logical address space into fixed-size pages and physical memory into equal-size frames. A page can be loaded into any free frame, so the process does not need to occupy contiguous physical memory. Paging mainly focuses on efficient physical-memory allocation and is widely used for virtual memory.

Segmentation divides a program into variable-sized logical units such as code, data, stack, and heap. Each segment represents a meaningful part of the program. Segmentation therefore provides a memory model that is closer to the programmer's logical view.

Memory allocation:

In paging, allocation is performed in fixed-size units. This makes allocation simple and avoids the need to find large contiguous regions. In segmentation, allocation is performed using variable-sized segments, so finding suitable free memory can be more complicated.

Fragmentation:

Paging eliminates external fragmentation because all frames have the same size. However, it may cause internal fragmentation when a page is not completely filled. Segmentation can suffer from external fragmentation because segments have different sizes, although it generally does not have the same fixed-block internal fragmentation problem.

Flexibility and protection:

Segmentation provides strong logical protection and sharing. For example, the code segment can be read-only, while the data segment can be read-write. Paging also supports protection, but its units are based on fixed-size pages rather than logical program components.

Performance:

Paging may require page-table lookups, but the TLB significantly reduces translation overhead. Segmentation requires segment-table lookups and bounds checking. In modern systems, paging is commonly preferred for physical-memory management because of its predictable allocation.

In summary, paging is efficient for physical-memory allocation and avoids external fragmentation, while segmentation provides better logical organization, protection, and sharing. Some systems combine both approaches so that segmentation provides the logical structure and paging manages physical memory efficiently.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured,	Organized and understandable	Limited clarity and structure	Poor clarity; difficult to

		and easy to understand	e presentation		understand
--	--	---------------------------	----------------	--	------------